

Final Project Guide: SQL to NoSQL Redesign

System: Sari-Sari Store POS & Inventory Management System

Database Chosen: MongoDB (Document-based NoSQL)

Stack: Node.js · Express · MongoDB · Mongoose · HTML/CSS/JS

How to use this guide: This file is your complete preparation kit. It contains your PDF documentation draft (Section 1), slide-by-slide talking points (Section 2), demo walkthrough (Section 3), Postman demo steps (Section 4), and anticipated professor Q&A (Section 5).

1. Documentation (Draft for your PDF)

1. Introduction

The Sari-Sari Store Point-of-Sale (POS) & Inventory System is a web-based application designed to help local store owners manage their inventory, process customer checkout, and view sales analytics. Originally conceptualized with a traditional relational database in mind, this project represents the redesign of the system into a NoSQL architecture to improve scalability, reduce read-latency during checkout, and simplify the data models.

2. Existing SQL System Overview

(Note: Since you built this in NoSQL, this section describes how it would have looked in SQL to satisfy the rubric). - **Purpose:** To manage users (cashiers/owners), track product inventory, and record sales transactions. - **Key Features:** Inventory management, shopping cart processing, receipt generation, and sales analytics. - **Sample SQL Tables (ERD Concept):** - **Users** (id, username, password, role) - **Products** (id, code, name, category, price, stock) - **Sales** (id, user_id, total, date) - **SaleItems** (sale_id, product_id, quantity, price_at_sale) (In SQL, generating a receipt requires joining **Sales**, **SaleItems**, and **Products**.)

3. NoSQL Design

- **Chosen NoSQL Type:** Document Store (MongoDB).

- **Design Philosophy:** Instead of normalizing data across multiple tables and relying on expensive `JOIN` operations, we embraced **Denormalization** and **Embedding**. In our NoSQL design, a `Sale` document embeds the complete array of items sold at that exact moment.

4. Data Model (JSON Examples)

User Collection (users):

```
{
  "_id": "64b1f...a34",
  "username": "Jumailah",
  "role": "cashier",
  "firstName": "Jumailah",
  "lastName": "Mamalampac",
  "status": "active"
}
```

Product Collection (products):

```
{
  "_id": "64b1f...b56",
  "name": "Lucky Me Pancit Canton",
  "category": "canned & instant goods",
  "price": 15,
  "quantity": 50,
  "emoji": "🍜",
  "image": "img/lucky-me.png"
}
```

Sale Collection (sales) — *Notice the Embedded Array:*

```
{
  "_id": "64b1f...c78",
  "soldBy": {
    "userId": "64b1f...a34",
    "username": "Jumailah",
    "fullName": "Jumailah Mamalampac"
  },
  "items": [
    {
      "productId": "64b1f...b56",
      "code": "64b1f...b56",
      "name": "Lucky Me Pancit Canton",
      "category": "canned & instant goods",
      "priceAtSale": 15,
      "quantity": 2,
      "subtotal": 30
    }
  ],
  "grandTotal": 30,
  "saleDate": "2026-05-20T10:00:00Z"
}
```

5. Implementation

(Include screenshots in your PDF of the following) - **Create (Insert)**: Show the "Add Product" modal or registration code. - **Read (Retrieve)**: Show the Cashier Dashboard displaying products or the Sales Analytics charts. - **Update**: Show the "Edit Product" modal updating price or stock. - **Delete**: Show the "Remove" button logic for deleting a product from inventory.

6. Performance & Security Discussion

Performance Considerations: - **No Joins Needed**: By embedding `saleItems` directly inside the `Sale` document, we can retrieve an entire receipt with a single database read `Sale.findById()`. In SQL, this would take multiple queries or complex joins. - **Historical Accuracy**: By snapshotting data (like `priceAtSale` and `name`) directly inside the embedded `items` array, our sales records remain historically accurate even if the main product document changes price later. This is much more efficient than creating complex historical price tracking tables in SQL.

Security Considerations: - **Authentication**: JWT (JSON Web Tokens) are used to secure API endpoints, ensuring only logged-in cashiers can process sales. - **Access Control**: Role-based access control (Admin vs. Cashier) prevents cashiers from accessing the owner's analytics dashboard or modifying inventory directly. - **Data Protection**: Passwords are mathematically hashed using `bcrypt`.

before being stored in the MongoDB database, meaning they cannot be read by anyone, not even the database administrator.

7. Conclusion

Redesigning the Sari-Sari Store system using MongoDB demonstrated that NoSQL document structures align perfectly with POS workflows. By prioritizing data embedding over normalization, the system achieved faster read speeds for sales analytics and a highly simplified backend architecture.

2. Presentation Outline (10-15 Minutes)

Slide 1: Title & Introduction (1 min)

- Introduce yourself and the system (Sari-Sari Store POS).
- Explain the core purpose: to manage inventory and sales for local stores.

Slide 2: The Old SQL Way (2 mins)

- Briefly show what the ERD would look like (Users -> Sales <- SaleItems -> Products).
- Point out the bottleneck: to view one complete transaction, the database has to join 3-4 different tables.

Slide 3: The NoSQL Redesign (3 mins)

- Introduce **MongoDB** (Document-based NoSQL).
- Show the **Sales** JSON document.
- **Key Talking Point:** Highlight the **Embedded Array**. Explain how taking the **SaleItems** and pushing them directly into the **Sale** document eliminated the need for joins entirely.

Slide 4: CRUD Operations (2 mins)

- Briefly talk about how you implemented Create, Read, Update, and Delete.
- Mention your use of Mongoose (Object Data Modeling for Node.js).

Slide 5: Performance & Security (3 mins)

- **Performance:** Explain "Data Snapshotting". How embedding the product name and price directly into the sale document saves time and protects history.

- **Security:** Mention `bcrypt` for password hashing and JWT for session management.

Slide 6: Challenges & Conclusion (2 mins)

- **Challenge Encountered:** Transitioning from relying on strict relational Foreign Keys to managing data consistency in application logic (e.g., deducting stock manually when a sale is processed).
- **Conclusion:** Why NoSQL was the perfect fit for this specific use-case.

3. Demo Preparation

- Have your Node server running (`npm run dev`).
- Open two browser windows: one logged in as an **Admin/Owner** (to show CRUD for products) and one logged in as a **Cashier** (to show adding items to cart and processing a sale).
- Be ready to open MongoDB Compass (or a similar viewer) to quickly show the professor how the JSON documents look live in the database.

4. Postman API Demo Guide

(Use this if asked to demonstrate CRUD via API without the browser UI)

Base URL: `http://localhost:3000/api`

Note: You need a JWT token from logging in. Run Login first, then copy the token and paste it in the Authorization header of all subsequent requests.

Step 1 — Admin Login (Get JWT Token)

- **Method:** `POST`
- **URL:** `http://localhost:3000/api/auth/admin-login`
- **Body (raw JSON):**

```
{
  "username": "your_admin_username",
  "password": "your_admin_password"
}
```

- Copy the `token` from the response. You'll use it as a Bearer Token for all other requests.
-

Step 2 — READ: Get All Products

- **Method:** `GET`
 - **URL:** `http://localhost:3000/api/products`
 - **Headers:** `Authorization: Bearer <paste_token_here>`
 - **Expected Result:** An array of all 25 product documents in JSON format.
-

Step 3 — CREATE: Add a New Product

- **Method:** `POST`
- **URL:** `http://localhost:3000/api/products`
- **Headers:** `Authorization: Bearer <paste_token_here>`
- **Body (raw JSON):**

```
{
  "name": "Kopiko 78°C Coffee",
  "category": "beverages",
  "price": 15,
  "quantity": 30,
  "emoji": "☕"
}
```

- **Expected Result:** `201 Created` with the new product document including its auto-generated `_id`.
-

Step 4 — UPDATE: Edit a Product

- **Method:** `PUT`
- **URL:** `http://localhost:3000/api/products/<product_id>` (copy an `_id` from the GET response)
- **Headers:** `Authorization: Bearer <paste_token_here>`
- **Body (raw JSON):**

```
{
  "name": "Kopiko 78°C Coffee",
  "category": "beverages",
  "price": 18,
  "quantity": 25
}
```

- **Expected Result:** `200 OK` with the updated product document showing the new price.
-

Step 5 — DELETE: Remove a Product

- **Method:** `DELETE`
 - **URL:** `http://localhost:3000/api/products/<product_id>`
 - **Headers:** `Authorization: Bearer <paste_token_here>`
 - **Expected Result:** `200 OK` with `{ "message": "Product deleted." }`.
-

5. Presentation Script (What to Say)

Opening (Slide 1 — 1 min)

"Good [morning/afternoon], everyone. Our project is the Sari-Sari Store POS and Inventory System. A Sari-Sari store is one of the most common small businesses in the Philippines — but most of them still track inventory and sales using pen and paper. Our goal was to redesign a SQL-based store system into a modern, flexible NoSQL application to solve real-world problems like slow reporting and rigid data structures."

SQL Overview (Slide 2 — 2 mins)

"If this system were built in a traditional relational SQL database, we would have at least four tables: Users, Products, Sales, and SaleItems. Every time a cashier processes a transaction, the database would need to JOIN across multiple tables just to display one receipt. As the store grows — more products, more cashiers, more transactions — those JOIN queries become increasingly slow and expensive."

NoSQL Redesign (Slide 3 — 3 mins)

"In our NoSQL redesign using MongoDB, we took a completely different approach. Instead of separating everything into tables, we use documents — which are essentially JSON objects. The most important design decision we made was embedding the sale items directly INSIDE the Sale document. This means retrieving a complete sales receipt is a single database operation, no joining required. We also snapshot the price at the time of sale inside each item, so even if we change a product's price tomorrow, historical receipts stay accurate forever."

CRUD Demo (Slide 4 — 2 mins)

"Let me quickly show you how Create, Read, Update, and Delete work in our system. [Switch to browser/Postman] We use Mongoose — which is an Object Data Modeling library for Node.js — to interact with MongoDB. Our API handles all four operations cleanly through RESTful endpoints."

Performance & Security (Slide 5 — 3 mins)

"On performance: our biggest gain is what we call data snapshotting. By embedding a full copy of the product info inside each sale, we avoid expensive lookups. For analytics — like our Top Products report — we only need to scan our Sales collection, not join three tables."

On security: we use bcrypt to hash all passwords before storing them — this means even if someone accessed our database, they could not read any passwords. We also use JWTs —

JSON Web Tokens — to authenticate every API call. And we have Role-Based Access Control — the Admin owner can do everything, but Cashiers are blocked from modifying inventory or viewing analytics."

Challenges & Close (Slide 6 — 2 mins)

"The biggest challenge we faced was data consistency. In SQL, Foreign Keys enforce relationships automatically — if you delete a product, the database prevents orphaned records. In MongoDB, we had to handle this in our application code manually — for example, deducting the stock count from the product document every time a sale is processed.

Overall, MongoDB was an excellent fit for this use case. The flexible schema allowed us to add fields like emoji and image to products without any migration headaches. Thank you."

6. Anticipated Professor Q&A

Question	Suggested Answer
<i>"Why did you choose MongoDB over Firebase?"</i>	MongoDB gives us full control over our data structure and runs locally or on the cloud. Firebase is excellent for real-time apps, but MongoDB with Mongoose is better suited for complex query analytics like our sales reporting.
<i>"How do you ensure data integrity without Foreign Keys?"</i>	We enforce referential integrity at the application level in our API routes. For example, our checkout route verifies that each product exists and has sufficient stock before deducting and saving the sale.
<i>"What happens if a product is deleted but appears in old sales records?"</i>	Because we embed a full snapshot of the product (name, price, category) directly inside each Sale document at checkout time, old sales are never affected by product changes or deletions.
<i>"What is the biggest disadvantage of this design?"</i>	Data duplication. Because we embed product data inside each Sale, the same product name may appear in thousands of documents. In SQL, we'd store it once and reference it. We accept this trade-off for much faster read performance.
<i>"How does bcrypt work?"</i>	bcrypt applies a one-way cryptographic hash function with a salt to the password. The result cannot be reversed — so to verify a login, we hash the entered password again and compare the two hashes.
<i>"What is a JWT?"</i>	A JSON Web Token is a digitally signed string that carries the user's identity and role. The server issues it on login. The client sends it with every request. The server verifies the signature to confirm the user is authenticated without checking a session database.